Twin Neighbor Embedding: A Pre-Embedding Transform for Improved Network Visualization.

Alexandros Athanasiadis

Nikos Pitsianis
Faculty Supervisor

Department of Electrical and Computer Engineering
Aristotle University of Thessaloniki

Abstract. This thesis aims to discuss challenges in existing methods for the visual analysis of networks and describe the Twin Neighbor Embedding (TNE) method, a simple pre-embedding transform designed to improve the quality of existing network layout algorithms, based on unpublished related research [1]. Network representations play a central role in scientific data analysis, as many real-world systems and relationships are naturally modeled as networks. Network visualization is therefore an integral component of network analysis, as spatial layouts can reveal structural properties of the underlying network in an intuitive and interpretable manner. However, existing network layout methods exhibit key limitations that can obscure structural relationships and hinder accurate interpretation of the data. SNE based methods have been applied in recent years to the layout of sparse networks using efficient and scalable algorithms [2]. However methods like SG-t-SNE still exhibit key limitations. We identify three recurring issues which the Twin Neighbor Embedding method aims to address: (a) vertices with small graph-theoretic distance or strong edge weights are often placed excessively close in the layout, leading to visual crowding and partial collapse of local substructure; (b) certain vertices or groups of vertices are erroneously placed close together, forming hallucinatory clusters that suggest relationships not present in the data; and (c) conventional layout methods do not explicitly encode geometric edge placement, which often results in excessive crossings of long edges. The Twin Neighbor Embedding method improves on these limitations by encoding vertex adjacency, edge adjacency, and vertex–edge incidence relationships in a unified augmented matrix used as a pre-layout transform. This representation is then supplied to standard layout algorithms in place of the traditional adjacency matrix. Despite its conceptual simplicity, the proposed transform improves layout quality across a variety of real-world and synthetic networks, as demonstrated through visual analysis and quantitative evaluation of neighborhood preservation, classification accuracy, vertex crowding and edge crossings.

1. Introduction

1.1. Motivation

Networks have been used and studied for centuries, as they can model a variety of systems and relationships. At their most basic, networks are a set of data points (called vertices or nodes) with interconnections between them (called edges or links). The versatility of networks enables us to gain insight into systems, prove conjectures, or develop algorithms in many scientific fields.

Network analysis and scientific network visualization are an increasingly vital part of data analytics for exploratory study and confirmatory analysis of relationships, patterns, community structures, and their evolution or dynamics in real-world data networks across various domains such as biology [3], [4], [5], [6], neurology [7], sociology [8], and related domains [9], [10].

1.2. Problem Description

A simple undirected graph G is the tuple (V, E) consisting of a set of vertices (nodes) V, and a set of edges (links) $E \subseteq V \times V$. Denote by $|V|$ the number of vertices and by $|E|$ the number of edges. If two vertices $v, u \in V$ form an edge $(v, u) \in E$ then we call v and u neighboring or adjacent, and we write $v \sim u$. A vertex v is incident to an edge e if it is part of e. Throughout this work we shall use the terms network and graph interchangeably.

Network visualization conventionally is the task of assigning each vertex $v \in V$ a position in some surface Σ. Then the edges may be drawn as lines or curves (not strictly in Σ) connecting their incident vertices. As a post-processing step, additional network properties and features may be overlaid on top of the layout as functions defined on the vertices or edges.

Consider the graph $G = (V, E)$. Each vertex $v \in V$ is mapped to a unique point $\mathbf{x}(v) \in \Sigma$. The set $X = \{\mathbf{x}(v) \mid v \in V\}$ is called the vertex layout or embedding[1] and is determined by a network layout algorithm.

The surface Σ is called the embedding space and is commonly $\mathbb{R}^2$, but sometimes $\mathbb{R}^3$ or even restricted to $\mathbb{S}^1$ or $\mathbb{S}^2$ [11], [12], [13]. For the purposes of visualization Σ is low-dimensional.

Existing software packages for scientific network visualization differ primarily in three aspects: (i) the choice of embedding space Σ, (ii) the choice of embedding criteria and layout algorithm, and (iii) post-processing utilities and overlays. The analysis in this thesis is primarily concerned with (ii), and from this point forward it will be assumed that $\Sigma \equiv \mathbb{R}^d$ (normally $d = 2$) and overlays will be used only to highlight structural aspects of the network useful in the visual understanding and qualitative assessment of the layout.

Constructing such embeddings is inherently challenging. A low-dimensional layout must represent a complex network topology while preserving structural relationships that are important for visual interpretation. Because many structural constraints cannot be simultaneously satisfied in a two-dimensional embedding, layout algorithms inevitably introduce distortions whose nature depends on the chosen embedding objective.

1.3. Related Work

Various layout algorithms exist with differing criteria for determining the spatial configuration X. Some of these methods and criteria will be discussed in more detail in Section 2.1 and Section 2.2.

A common criterion is to preserve the pairwise geodesic distances of vertices on G proportionally in the layout X. Yet it is impractical to rigidly impose all geodesic distance pairs be preserved in equal measures, considering the confined low-dimensional embedding space does not allow for the variety, irregularity, and uncertainty in sparse data networks. We prioritize that adjacent vertices in network G are placed in close proximity in the layout X. This is the Stochastic Neighbor Embedding (SNE) [14] principle applied to network topology.

However SNE-based layouts suffer from undesirable artifacts that obscure structural relationships and hinder accurate interpretation of the data. We identify three recurring issues:

[1] In topological graph theory an embedding strictly is a drawing of the vertices and edges in a way such that the edges may intersect only at their ends in the target embedding space. In this thesis embedding and layout are used interchangeably, in all cases meaning a layout.

(a) vertex crowding: vertices with small graph-theoretic distance or strong edge weights are often placed excessively close in the layout, leading to visual crowding and partial collapse of local substructure.
(b) erroneous clustering: certain vertices or groups of vertices are erroneously placed close together, forming hallucinatory clusters that suggest relationships not present in the data.
(c) excessive long-edge crossings: conventional layout methods do not explicitly encode geometric edge placement, which often results in excessive crossings of long edges.

t-SNE [15] was largely successful in alleviating the crowding issue compared to classical SNE for dimensionality reduction of point-cloud data. SG-*t*-SNE [2], which adapts *t*-SNE for sparse networks, has achieved good performance, while still exhibiting some of the limitations of SNE methods.

In an effort to address the remaining issues of SG-*t*-SNE for sparse networks we developed the Twin Neighbor Embedding (TNE) method and the TWIN software package as part of yet unpublished research with Dimitris Floros, Nikos Pitsianis, and Xiaobai Sun [1].

AA1: Section break here?

The aim of this thesis is to present and analyze the TNE method and the TWIN software and compare it to existing network layout methods in terms of visual quality and quantitative evaluation criteria. Additional contributions are presented in Section 1.4.

TNE is not a layout algorithm in the traditional sense, but instead a pre-layout transform of G which constructs a supra-network consisting of G, its line graph G_ℓ, and additional connections linking vertices of G to corresponding incident edges represented in G_ℓ. This is represented algebraically via a unified augmented matrix which encodes vertex and edge adjacency as well as vertex–edge incidence. This representation is then used in place of the traditional adjacency matrix to extend and improve pre-existing layout methods, primarily —but not limited to— SG-*t*-SNE. The method is described in detail in Section 4.

Despite its conceptual and mathematical simplicity, TNE proves effective in mitigating vertex crowding and erroneous clustering across various real-world and synthetic networks. Additionally, due to its explicit geometric encoding of edge placement it significantly alleviates the problem of excessive edge crossing. Various visualization results, as well as quantitative evaluations of the method are presented in Section 5.

Our software package TWIN allows the user to interact with the Twin Neighbor Embedding method and compare to traditional vertex embedding in an interactive GUI with various tuning parameters. The package is open source and the source code can be found at https://github.com/alex-unofficial/twin-neighbor-embedding. The TWIN software package is discussed in Section 6.

Finally, we discuss the limitations of our method as well as potential future improvements in Section 7.

1.4. Contributions

AA2: Curved edge drawing based on edge layout

AA3: Multi-level approach(?) — see Section 2.1.1: Spring-Electrical model

AA4: We'll see...

2. Background

Network diagrams have been used for centuries to analyze complex relationships, appearing as early as the 14th century [16], long before the formal development of graph theory.

Early work in graph theory was closely related to properties of drawings of graphs, such as planarity — the ability to represent a graph in the Euclidean plane without intersecting edges. Investigations of polyhedral graphs by Leonhard Euler, particularly his formula relating vertices, edges, and faces of convex polyhedra, are often cited among the earliest results connected to graph theory.

Prior to the development of computer graphics, graph drawings in the form of node–link diagrams were created by manually placing each vertex and connecting them with lines or arrows (depending on the type of graph). In most cases the effective "layout algorithm" was simply the intuition of the artist drawing the diagram, placing related vertices closer together, or arranging them in a visually meaningful manner. However, more sophisticated layout methods were occasionally used such as circular layouts appearing in the combinatorial diagrams of Ramon Llull or the top-down arrangement of medieval family-tree drawings.

AA5: Too much historical context?

While the history of graph drawing spans several centuries, the modern field of network visualization largely emerged alongside the development of computer graphics in the 20th century.

The need to represent larger, more complex networks created the demand for automatic layout generation methods. Such methods vary widely in both methodology and objectives, depending on the type of input graph, the specific use case, and the scientific intent of the user.

We restrict our scope to scientific network visualization of general undirected sparse networks. Our aim is the analysis of networks for exploratory study and visual confirmation of community structures and relationships in data. Consequently we restrict our attention to unconstrained euclidean layouts of general networks. Specialized layouts designed for specific application domains and particular graph classes —such as grid layouts for VLSI and PCB design, tree layouts for hierarchical data networks, or spatially constrained layouts like circular layouts and arc diagrams— are outside the scope of this work.

2.1. Conventional Methods

Consider a graph $G = (V, E)$. We must assign to each vertex $v \in V$ a position $\mathbf{x}(v) \in \mathbb{R}^d$.

Denote by $n = |V|$ the number of vertices and by $m = |E|$ the number of edges. Given some ordering of the vertices $1, 2, ..., n$ the layout X is made of n position vectors $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n$ such that $\mathbf{x}_i = \mathbf{x}(i)$ is the position of vertex i. From a computational perspective the layout $X \in \mathbb{R}^{n \times d}$ can be treated as a conventional matrix whose rows are the vectors $\mathbf{x}_i^T$.

It is obvious that any single graph has infinitely many layouts, given we have not stated any constraints on $\mathbf{x}(v)$. However, not every layout is of the same quality. Take, for example, n randomly sampled positions in $\mathbb{R}^d$ and assign each one to a vertex. This is a valid layout, but it encodes no information about the underlying network. From a data analysis perspective, it is practically useless.

Therefore we must define what makes a quality layout. This is not a simple question, and existing methods adopt different perspectives.

Although layout algorithms are presented in many different forms, a large portion of the literature can be interpreted through the lens of optimization. Many methods explicitly define an objective or energy function that measures the quality of a layout and then apply numerical procedures to approximately minimize it. Other methods are not explicitly presented as optimization procedures, and may describe the algorithm in isolation — as analogues of physical systems, heuristic approaches, or functions of algebraic structures associated with the graph — but can still be understood as implicitly optimizing a related criterion. It is useful in these cases, when possible, to identify the implicit objective or criteria of a method in order to properly understand its behavior and results.

In the following sections we briefly review several widely used classes of layout algorithms, focusing on representative methods and the principles that underlie their formulations.

AA6: [17] is a good resource

2.1.1. Force-directed methods

Force-directed (or force-based) methods are among the most widely used approaches for scientific network visualization. The vast majority of algorithms implemented in academic and commercial network visualization software are based on this principle. Consequently, force-based methods exhibit substantial variation in theoretical framing, motivation, and execution.

The central idea is to treat each vertex v as a particle located at position $\mathbf{x}(v)$ analogous to a particle in a physical system. Forces are then defined between particles corresponding to the graph topology and relative position. The specific form of the forces varies between methods, but in general they induce an energy function or dynamical system whose equilibrium configurations correspond to the layout X. Methods in this category differ primarily in three aspects:

1. *The specific form of the forces and the corresponding energy function.*
   Different models define interactions between vertices in different ways. For example, spring-electrical methods apply attractive forces along edges and repulsive forces between all vertices, while stress and strain models define spring-like forces between all vertex pairs with natural lengths proportional to their geodesic distance in G.

2. *The mathematical approach used to obtain the equilibrium configuration.*
   Equilibrium configurations may be obtained by direct physical simulation of the dynamical particle system, wherein each particle's position is iteratively updated according to the forces acting upon it, or by numerical methods that compute equilibrium states of the system, for example by solving the corresponding equilibrium equations or by minimizing the associated energy function.

3. *The computational algorithms used to calculate the resulting layout.*
   Among the many force-based layout methods that exist, many differ primarily in the implementation strategies, particularly in techniques used to improve scalability and efficiency. This is especially important for large networks, where naive implementations tend to become impractically slow and memory inefficient. Common approaches include

approximation techniques, multilevel methods, and data structures that accelerate force computation.

The general framework described above gives rise to numerous concrete layout models. In the following sections several representative approaches are presented, beginning with the classical spring–electrical model and followed by methods based on alternative energy formulations such as LinLog and Multidimensional Scaling (MDS). For each method we briefly describe the force or energy model and the algorithm used to compute the layout.

The Spring-Electrical model. This model was first introduced by Peter Eades in 1984 [18]. In this first approach, each edge in G was modeled as a spring whose attractive force grew logarithmically with distance, while non-adjacent vertices were subject to a repulsive force inversely proportional to their distance. A physical simulation was then used to find a stable layout starting from an initial configuration.

Later, Fruchterman and Reingold [19] suggested a model in which attractive spring forces are proportional to the squared distance between adjacent vertices, while repulsive electrical forces act between all vertex pairs and are inversely proportional to their distance.

Let the unit direction vector from j to i be defined as

$$\hat{\mathbf{r}}_{ij} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|} \tag{1}$$

The attractive force is

$$\mathbf{F}_a(i,j) = -\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{K}\,\hat{\mathbf{r}}_{ij} \quad i \sim j \tag{2}$$

and the repulsive force is

$$\mathbf{F}_r(i,j) = \frac{K^2}{\|\mathbf{x}_i - \mathbf{x}_j\|}\,\hat{\mathbf{r}}_{ij} \quad i \neq j \tag{3}$$

where the parameter K represents the desired edge length in the final layout.

These forces correspond to the following energy function [20]:

$$\mathcal{E}_{\mathrm{FR}}(X) = \sum_{i\sim j} \frac{1}{3K}\,\|\mathbf{x}_i - \mathbf{x}_j\|^3 - \sum_{i<j} K^2 \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{4}$$

The layout is computed iteratively. For each vertex i, the normalized force direction $\hat{\mathbf{f}}_i$ is calculated as:

$$\mathbf{F}_i = \sum_{j\sim i} \mathbf{F}_a(i,j) + \sum_{j\neq i} \mathbf{F}_r(i,j), \quad \hat{\mathbf{f}}_i = \frac{\mathbf{F}_i}{\|\mathbf{F}_i\|} \tag{5}$$

Using the previous configuration $X^{(j)}$, the vertex position is updated by moving in the direction of $\hat{\mathbf{f}}_i$ with decreasing step length $h^{(j)}$

$$\mathbf{x}_i^{(j+1)} \leftarrow \mathbf{x}_i^{(j)} + h^{(j)}\,\mathbf{f}_i \tag{6}$$

until the layout stabilizes at an equilibrium point. Later refinements introduce adaptive step-length schemes [21], [22].

Despite the physical interpretation, the algorithm does not simulate particle dynamics according to classical mechanics. Instead it behaves as a heuristic gradient-based optimization procedure for minimizing (4).

This method usually works well for small graphs, but for larger graphs it is prone to becoming trapped in local minima of the energy function [17]. In addition, the algorithm itself must calculate repulsive interactions between all $\frac{n(n-1)}{2}$ vertex pairs, resulting in $\mathcal{O}(n^2)$ computational complexity.

Fast force approximation. The time complexity can be improved by using a spatial partitioning technique such as the Barnes-Hut algorithm [23], widely used in physics for n-body simulations, which can approximate the forces in $\mathcal{O}(n \log n)$ time with good accuracy. Such an approach is taken in [24], [25] and [22].

Multilevel Methods. To address the problem of the solution becoming trapped in local minima, various algorithms **AA7:** (citations) utilize a multilevel (or multiscale) approach by generating a progressively coarser series of graphs $G = G^0, G^1, G^2, ..., G^k$, such that G^{j+1} is a coarse approximation of G^j with fewer vertices, while preserving the basic connectivity structure. Then a progressively finer series of layouts $X_k, X_{k-1}, X_{k-2}, ..., X_0 = X$ is generated, where X_j is a layout of graph G^j, using $X_j^{(0)} = X_{j+1}$ as its initial configuration. The core idea is that by having fewer vertices that encode the same basic connectivity, the energy function $\mathcal{E}^j$ will correspond to a coarse approximation of the original $\mathcal{E}^0$, while being smoother and therefore easier to traverse without becoming trapped in local minima. At the same time, since $\mathcal{E}^j$ has the same basic shape as $\mathcal{E}^{j-1}$, its minimum is likely to be close to a minimum of the finer energy function. By repeating this process in finer and finer steps, the method attempts to refine the computed equilibrium point for progressively finer versions of G. The final layout X_0 is an approximate equilibrium configuration of $\mathcal{E}^0$, but is less likely to lie in one of its many local minima compared to random initialization. Notably this approach is not limited to any specific model, or to network layout in general, but has found wide application in a variety of combinatorial optimization problems **AA8:** (more citations) .

AA9: We should consider a multi-level approach to TNE as well. A refinement strategy like the one described above could produce better results, but there is an even simpler idea: first generate a traditional vertex layout using SG-t-SNE and generate the edge layout as defined in the paper, then use the combined vertex–edge layout as the initial configuration for the TNE step. This might be able to retain some wanted properties of the vertex layout while alleviating crowding. Will test soon.

LinLog and r-PolyLog models. The Fruchterman-Reingold model tends to produce layouts with small and uniform edge lengths and few edge crossings. However, it performs poorly at separating clusters, since edges connecting different clusters should typically be longer [20].

The LinLog energy model [20] was introduced by Andreas Noack in 2004, to improve on the cluster separation problem of the Fruchterman-Reingold force model. Noack takes an energy-centric approach to defining the dynamical system of particles that represent the layout X.

Specifically the LinLog energy function $\mathcal{E}_{\text{LinLog}}$ is defined as

$$\mathcal{E}_{\text{LinLog}}(X) = \sum_{i \sim j} \|\mathbf{x}_i - \mathbf{x}_j\| - \sum_{i<j} \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{7}$$

and the layout problem is framed from the explicit perspective of finding a minimal-energy configuration X.

A specific algorithm to find the minima is not described, but standard optimization methods in conjunction with multilevel approaches may be used in practice.

LinLog has better separation of clusters with small diameter compared to Fruchterman-Reingold, but sacrifices edge length uniformity as a result.

Noack also introduces the generalized r-PolyLog class of energy models with corresponding energy functions

$$\mathcal{E}_{r\text{-PolyLog}}(X) = \sum_{i \sim j} \frac{1}{r} \|\mathbf{x}_i - \mathbf{x}_j\|^r - \sum_{i<j} \log\|\mathbf{x}_i - \mathbf{x}_j\| \tag{8}$$

Where parameter $r \geq 1$ describes the model behaviour. Choice $r = 1$ separates clusters while larger values of r increasingly favor uniform edge lengths.

Notice that 1-PolyLog is equivalent to LinLog (7), while the 3-PolyLog energy is the same as Fruchterman-Reingold (4), up to a constant scaling factor.

The Stress model. A limitation of Spring-Electrical models like those described above is that they primarily enforce local edge length constraints and do not explicitly encode global distance relationships between vertices. Real-world networks often include quantitative information on their edges in the form of weights representing similarity or distance between the vertices. While such information could be incorporated by adjusting the strength of attractive forces, the Stress model instead adopts a more principled approach that attempts to match the distances between all vertex pairs.

Consider a weighted graph $G = (V, E, w)$ where V and E are the vertex and edge sets, and $w : E \to W$ maps each edge e to a value $w(e) \in W$. We denote edges by $e_{ij} = (i, j)$ and write $w_{ij} = w(e_{ij})$ when $e_{ij} \in E$.

For each vertex pair (i, j) we derive a target distance $d_{ij} \in \mathbb{R}^+$. If $e_{ij} \in E$, the distance is obtained by applying a transformation $f : W \to \mathbb{R}^+$ that maps each edge weight to a distance, so that $d_{ij} = f(w_{ij})$. For non-adjacent vertices, d_{ij} is typically defined as the shortest-path distance between i and j in G when edges are assigned lengths $f(w_{ij})$.

We assume that G is connected, ensuring that d_{ij} is defined for all vertex pairs. If the graph is disconnected, its connected components can be identified and the algorithm applied to each component separately.

The choice of transformation f depends on the information represented by the weights. If the weights encode distances between vertices, the identity mapping $f(w) = w$ suffices. If instead w_{ij} represents a similarity or probability, a natural choice is $f(w) = -\log(w)$. This transformation preserves the ordering of similarities and has the useful property that shortest-path distances become additive: probabilities along a path multiply while their negative logarithms sum.

Once the ideal distances d_{ij} are defined for all vertex pairs, the goal is to compute a layout that approximates the ideal distances as closely as possible.

In the stress model, each vertex pair is connected by a spring with nominal length d_{ij} and stiffness k_{ij}. This yields the energy function

$$\mathcal{E}_{\text{Stress}}(X) = \sum_{i<j} k_{ij}\big(\|\mathbf{x}_i - \mathbf{x}_j\| - d_{ij}\big)^2 \tag{9}$$

whose minimum corresponds to the layout that optimally matches the ideal distances. A typical choice is $k_{ij} = 1/d_{ij}^2$ which yields the equivalent formulation

$$\sum_{i<j} \left(\frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{d_{ij}} - 1 \right)^2 \tag{10}$$

thereby measuring the relative deviation between the actual and ideal edge lengths.

Although the stress model is often grouped with force-directed methods, it is more naturally interpreted as an explicit optimization formulation derived from Multidimensional Scaling (MDS) [26], [27]. It was introduced to network layout by Kamada and Kawai in 1989 [28], who proposed minimizing (9) using Newton's method. More recently, stress majorization [29] has become the preferred approach due to its robustness and guaranteed monotonic decrease of the stress function.

The Stress model as defined is not easily scalable for large networks. Constructing the model requires computing shortest-path distances for all vertex pairs. Using Johnson's algorithm this step requires $\mathcal{O}(n^2 \log n + nm)$ time and $\mathcal{O}(n^2)$ memory to store the computed distances. The stress-majorization step additionally requires solving dense linear systems in each iteration, which can be more computationally expensive than the distance computation itself. Consequently, for very large graphs the method becomes both computationally expensive and memory intensive [17]. Various optimizations and approximations have therefore been proposed that trade accuracy for reduced time and memory requirements. **AA10:** Citations?

2.1.2. Spectral Methods

Spectral layout for network visualization was introduced by Kenneth Hall in 1970 [30] and uses the spectral properties of the graph Laplacian to determine vertex positions.

In Hall's original formulation we seek to minimize the energy function

$$\mathcal{E}_{\text{Spectral}}(X) = \sum_{i \sim j} w_{ij} \, \|\mathbf{x}_i - \mathbf{x}_j\|^2 \tag{11}$$

where w_{ij} denotes the weight of edge $e_{ij} \in E$ and $w_{ij} = 0$ if $e_{ij} \notin E$. For undirected graphs $w_{ij} = w_{ji}$.

This objective penalizes large distances between adjacent vertices, weighted by their edge weights. To avoid the trivial solution $\mathbf{x}_i = \mathbf{x}_j$ for all vertices, we impose orthogonality and normalization constraints $X^T X = I$ and $X^T \mathbf{1} = 0$.

The weighted graph Laplacian is defined as

$$L^w = D - W \tag{12}$$

where D is the diagonal weighted degree matrix defined as $D_{ii} = \sum_{j \neq i} w_{ij}$ and $D_{ij} = 0$ for $i \neq j$, while W is the weighted adjacency matrix $W_{ij} = w_{ij}$ when $i \neq j$ and $W_{ii} = 0$.

Using the laplacian definition in (12), the energy (11) can be written as

$$\mathcal{E}_{\text{Spectral}}(X) = \text{tr}(X^T L^w X), \quad \text{subject to } X^T X = I \text{ and } X^T \mathbf{1} = 0 \tag{13}$$

Minimizing (13) subject to $X^T X = I$ and $X^T \mathbf{1} = 0$ yields solutions X^* whose columns are eigenvectors X_k^* of L^w. The energy at the optimum becomes

$$\mathcal{E}_{\text{Spectral}}(X^*) = \sum_{k=1}^{d} \lambda_k \tag{14}$$

where λ_k are the corresponding eigenvalues. For a connected graph, the smallest eigenvalue of L^w is 0 with eigenvector $\mathbf{1}/\sqrt{n}$. The constraint $X^T \mathbf{1} = 0$ excludes this trivial solution, therefore the optimal layout is obtained by selecting the d smallest non-zero eigenvalues and their corresponding eigenvectors.

The spectral layout energy function penalizes large distances between adjacent vertices, particularly those connected by strong edge weights, producing layouts that vary smoothly over the graph. Vertices with many or strong edges tend to be placed near each other, while weak connections correspond to larger separations.

However, the spectral layout method often produces less interpretable results for many complex real-world graphs. High-degree vertices tend to collapse toward the center, vertex crowding is common, and the layout may appear stretched along the eigenvector directions. Consequently, spectral layouts are rarely used as final visualizations and instead commonly serve as initial configurations for other layout algorithms.

Normally, a full eigen-decomposition requires $\mathcal{O}(n^3)$ time. However, the layout requires computing only the d smallest non-zero eigenvectors, so iterative eigensolvers can approximate the solution very efficiently even for very large graphs [31].

2.1.3. Stochastic methods

AA11: SNE and t-SNE as embedding methods for kNN graphs

2.2. State of the Art

AA12: SG-t-SNE, ForceAtlas2, Yifan Hu, OpenOrd, LinLog, NEATO, PivotMDS, GRIP, etc.

AA13: Network visualization software packages: Cytoscape, Gephi, Graphviz, Tulip

3. Layout evaluation and metrics

AA14: Description of visual examination of layouts and metrics for layout performance

4. The Twin Neighbor Embedding method

AA15: The method, how and why it works.

5. Results

AA16: Including visualization results as well as computed metrics

6. The TWIN software package and GUI

AA17: Brief description of the Software, the parameters and the GUI

7. Conclusion and Limitations

AA18: What the method improves on, where it doesn't, and how it may be improved in the future.

Bibliography

[1] A. Athanasiadis, D. Floros, N. Pitsianis, and X. Sun, "Twin Neighbor Embedding of Sparse Data Networks," 2025.

[2] N. Pitsianis, A.-S. Iliopoulos, D. Floros, and X. Sun, "Spaceland Embedding of Sparse Stochastic Graphs," in *IEEE HPEC*, 2019, pp. 1–8. doi: 10.1109/HPEC.2019.8916505.

[3] K. Melnyk, K. Weimann, and T. O. F. Conrad, "Understanding Microbiome Dynamics via Interpretable Graph Representation Learning," *Scientific Reports*, vol. 13, no. 1, p. 2058, Feb. 2023, doi: 10.1038/s41598-023-29098-7.

[4] H. Ashoor *et al.*, "Graph Embedding and Unsupervised Learning Predict Genomic Sub-Compartments from HiC Chromatin Interaction Data," *Nature Communications*, vol. 11, no. 1, p. 1173, Mar. 2020, doi: 10.1038/s41467-020-14974-x.

[5] F. Costa, D. Grün, and R. Backofen, "GraphDDP: A Graph-Embedding Approach to Detect Differentiation Pathways in Single-Cell-Data Using Prior Class Knowledge," *Nature Communications*, vol. 9, no. 1, p. 3685, Sept. 2018, doi: 10.1038/s41467-018-05988-7.

[6] U. Stelzl *et al.*, "A Human Protein-Protein Interaction Network: A Resource for Annotating the Proteome," *Cell*, vol. 122, no. 6, pp. 957–968, Sept. 2005, doi: 10.1016/j.cell.2005.08.029.

[7] H.-J. Park and K. Friston, "Structural and Functional Brain Networks: From Connections to Cognition," *Science*, vol. 342, no. 6158, p. 1238411, Nov. 2013, doi: 10.1126/science.1238411.

[8] C. Kadushin, Ed., *Understanding Social Networks*. New York: Oxford University Press, 2012.

[9] V. Filipov, A. Arleo, and S. Miksch, "Are We There yet? A Roadmap of Network Visualization from Surveys to Task Taxonomies," *Computer Graphics Forum*, vol. 42, no. 6, p. e14794, 2023, doi: 10.1111/cgf.14794.

[10] C. V. R. Hütter, C. Sin, F. Müller, and J. Menche, "Network Cartographs for Interpretable Visualizations," *Nature Computational Science*, vol. 2, no. 2, pp. 84–89, Feb. 2022, doi: 10.1038/s43588-022-00199-z.

[11] D. J. Watts and S. H. Strogatz, "Collective Dynamics of 'Small-World' Networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998, doi: 10.1038/30918.

[12] J. Miller, V. Huroyan, and S. Kobourov, "Balancing Between the Local and Global Structures (LGS) in Graph Embedding," *Graph Drawing and Network Visualization*, vol. 14465. Springer Nature Switzerland, Cham, pp. 263–279, 2023. doi: 10.1007/978-3-031-49272-3_18.

[13] Y. Lu, J. Corander, and Z. Yang, "Doubly Stochastic Neighbor Embedding on Spheres," *Pattern Recognition Letters*, vol. 128, pp. 100–106, Dec. 2019, doi: 10.1016/j.patrec.2019.08.026.

[14] G. E. Hinton and S. Roweis, "Stochastic Neighbor Embedding," in *Advances in Neural Information Processing Systems*, S. Becker, S. Thrun, and K. Obermayer, Eds., MIT Press, 2002.

[15] L. van der Maaten and G. Hinton, "Visualizing Data Using T-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[16] D. E. Knuth, "Two thousand years of combinatorics," in *Combinatorics: Ancient and Modern*, Oxford University Press, 2013, pp. 3–37.

[17] Y. Hu and L. Shi, "Visualizing large graphs," *WIREs Computational Statistics*, vol. 7, no. 2, pp. 115–136, 2015, doi: https://doi.org/10.1002/wics.1343.

[18] P. Eades, "A Heuristic for Graph Drawing," 1984. [Online]. Available: https://api.semanticscholar.org/CorpusID:63936426

[19] T. M. J. Fruchterman and E. M. Reingold, "Graph drawing by force-directed placement," *Software: Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991, doi: https://doi.org/10.1002/spe.4380211102.

[20] A. Noack, "An Energy Model for Visual Graph Clustering," in *Graph Drawing*, G. Liotta, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 425–436.

[21] I. Bruß and A. Frick, "Fast Interactive 3-D Graph Visualization," in *International Symposium Graph Drawing and Network Visualization*, 1995. [Online]. Available: https://api.semanticscholar.org/CorpusID:16094190

[22] Y. Hu, "Efficient, High-Quality Force-Directed Graph Drawing," *The Mathematica journal*, vol. 10, pp. 37–71, 2006, [Online]. Available: https://api.semanticscholar.org/CorpusID:14599587

[23] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," *\nat*, vol. 324, no. 6096, pp. 446–449, Dec. 1986, doi: 10.1038/324446a0.

[24] D. Tunkelang, "A Numerical Optimization Approach to General Graph Drawing," p. , 1999.

[25] A. J. Quigley, "Large scale relational information visualization, clustering, and abstraction," PhD Thesis, 2001.

[26] W. S. Torgerson, "Multidimensional Scaling: I. Theory and Method," *Psychometrika*, vol. 17, no. 4, pp. 401–419, 1952, doi: 10.1007/BF02288916.

[27] J. B. Kruskal, "Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis," *Psychometrika*, vol. 29, pp. 1–27, 1964, [Online]. Available: https://api.semanticscholar.org/CorpusID:48165675

[28] T. Kamada and S. Kawai, "An algorithm for drawing general undirected graphs," *Information Processing Letters*, vol. 31, no. 1, pp. 7–15, 1989, doi: https://doi.org/10.1016/0020-0190(89)90102-6.

[29] E. Gansner, Y. Koren, and S. North, "Graph Drawing by Stress Majorization," 2004, p. . doi: 10.1007/978-3-540-31843-9_25.

[30] K. M. Hall, "An r-Dimensional Quadratic Placement Algorithm," *Management Science*, vol. 17, no. 3, pp. 219–229, Nov. 1970, doi: 10.1287/mnsc.17.3.219.

[31] Y. Koren, L. Carmel, and D. Harel, "ACE: A Fast Multiscale Eigenvectors Computation for Drawing Huge Graphs," 2002, pp. 137–144. doi: 10.1109/INFVIS.2002.1173159.